

Übungsblatt 5

Vorlesung „Secure Software Systems“ Wintersemester 2025/26

Praktische Übung Control-Flow Integrity and Data-Oriented Attacks

Bending Control-Flow Integrity

In dieser Aufgabe müssen Sie sich mit Binaries beschäftigen, die mit aktuellen *Control-Flow Integrity* Mechanismen kompiliert wurden. Diese Aufgabe soll zeigen, dass trotz aktiven Verteidigungen gewisse Angriffe trotzdem möglich sind. Um den Stack zu schützen wurde das Programm mit dem *Safe Stack* kompiliert, um Return-Adressen und lokale Variablen vor Buffer Overflows auf dem Stack zu schützen. Dieser Mechanismus schützt die Backward Edge im Kontext von CFI und ist stärker als z.B. der Stack Canary Mechanismus. Es gibt in dieser Aufgabe zwei Varianten des gleichen Programms: `bend` und `bend.cfi`. Beide sind funktional identisch, `bend.cfi` wurde jedoch mit aktiviertem *Forward-Edge Control-Flow Integrity* kompiliert. Im Speziellen wurde das in *LLVM* bzw. *clang* integrierte Typen-basierte CFI verwendet. Indirekte Sprünge können hier nur mehr Funktionen mit gleichen Typ-Signaturen anspringen.

Bei beiden Varianten, mit und ohne CFI, ist es möglich das Programm, durch Memory Corruption Angriffe dazu zu bringen, die Datei `flag.txt` auszulesen (bzw. jede andere beliebige Datei auf dem System). Das Ziel dieser Aufgabe ist es einen Exploit zu schreiben, der alle Verteidigungen, inklusive CFI, umgeht. Die Buffer Overflow Schwachstelle in diesem Programm resultiert aus der Verwendung der Funktion `strcat`, welche zwei C-Strings konkateniert und in den ersten String speichert. Ähnlich zu `strcpy` und `gets`, wird dieser Funktion die Länge des Ziel-Strings nicht mitgegeben und es findet kein bounds-checking statt. Dadurch entsteht dann eine Buffer Overflow Schwachstelle. Mittels dieses Buffer Overflows können Sie Funktionspointer überschreiben, die sich im gleichen Objekt befinden.

Aufgabe 1.

- i) Welchen Input müssen Sie schicken, um den Funktionspointer `print` im Objekt `String` zu überschreiben? Überschreiben Sie den Funktionspointer `print` mit dem Wert von `system` und bringen Sie das Programm dazu, diesen zu verwenden. Schreiben Sie das Kommando, also den ersten Parameter für `system`, in den Buffer des `String` Objektes. Erklären Sie, wieso `system` dann einen validen Parameter übergeben bekommt. Erweitern Sie das Exploit Template `exploit_normal.py` zu einem funktionierenden Exploit, der über `system` und die entsprechenden Kommandos die Datei `flag.txt` ausliest.
- ii) Kopieren Sie Ihren Code aus der vorherigen Teilaufgabe in die Datei `exploit_cfi.py`. Stellen Sie sicher, dass das richtige Programm, `bend.cfi`, ausgeführt wird. Ihr Exploit wird nun aufgrund von aktiviertem CFI nicht mehr funktionieren. Erklären Sie welche CFI Policy umgesetzt wird und warum diese Ihren Exploit verhindert. Analysieren Sie den CFI Check mittels *Cutter* oder *gdb* und erklären Sie die relevanten Assembly Instruktionen.

- iii) In dem Programm befinden sich noch weitere Funktionspointer, die Sie überschreiben können. Analysieren sie die verfügbaren Funktionen und die Funktionspointer. Finden Sie einen Funktionspointer, den Sie überschreiben können, und eine Zielfunktion, welche die Datei *flag.txt* ausliest. Kopieren sie ihren bisherigen Exploit und modifizieren Sie `exploit_cfi.py` so, dass der Exploit wieder funktioniert.

Nützliche Links zu clang CFI. Die relevante Policy im LLVM CFI für diese Übung ist *Indirect Function-Call Checks*.

- <https://clang.llvm.org/docs/ControlFlowIntegrity.html>
- <https://blog.trailofbits.com/2016/10/17/lets-talk-about-cfi-clang-edition/>
- Paper: On the Effectiveness of Type-based Control Flow Integrity <https://arxiv.org/pdf/1810.10649.pdf>
- Paper: Enforcing Forward-Edge Control-Flow Integrity in GCC & LLVM <https://www.usenix.org/system/files/conference/usenixsecurity14/sec14-paper-tice.pdf>

Von Use-After-Free zu Arbitrary Read/Write

Das Programm `uadop` implementiert eine einfache Text-basierte User Verwaltung. Mehrere Aktionen, wie “create”, “edit”, “delete”, “print” etc., die jeweils User Objekte verwalten, werden vom Programm unterstützt. Die User Objekte werden auf dem Heap angelegt. Außerdem hat das Programm eine Use-After-Free Schwachstelle. Diese Schwachstelle kann von einem Angreifer verwendet werden, um ein *Arbitrary Read/Write* Primitiv zu erhalten. Das Ziel ist es Zugriff zu dem Admin Menü (= einer System-Shell) zu erhalten. Jedoch ist das Admin Menü deaktiviert und durch ein Passwort geschützt. Um Zugriff zu erhalten, muss zuerst mit Hilfe des Arbitrary Read/Write, das Admin Passwort ausgelesen werden und dann das Admin Menü aktiviert werden.

Um die Analyse des Programms zu vereinfachen, gibt das Programm Informationen zu den Aufrufen von Funktionen zur Heapverwaltung aus (`malloc`, `realloc` und `free`). Die Ausgaben sind mit dem Präfix `[DEBUG]` markiert.

Aufgabe 2.

- Wo befindet sich die Schwachstelle in diesem Programm? Was ist das Problem? In welcher Reihenfolge müssen Sie welche Aktionen durchführen, damit ein *dangling pointer* von dem Programm verwendet wird? Wie könnte man die Schwachstelle einfach beheben?
- Bei der Use-After-Free Schwachstelle verwendet das Programm ein Objekt *U*, welches allerdings schon freigegeben wurde. Wie können Sie ein anderes Objekt (z.B. einen String) an derselben Stelle wie *U* anlegen? Wie sieht ein Objekt vom Typ `user_t` im Speicher aus? Welche Aktionen mit welchem Input werden benötigt, um das freigegebene User Objekt *U* mit Angreifer kontrollierten Werten zu überschreiben?
- Verwenden Sie die Use-After-Free Schwachstelle, um ein Arbitrary Read/Write zu erhalten. Wie können Sie die Schwachstelle nutzen, um das Admin Passwort auszulesen? Wie können Sie die Schwachstelle nutzen, um das Admin Menü zu aktivieren, also auf die Variable `admin_enabled` zu schreiben? Erweitern Sie das Exploit-Template zu einem funktionierenden Angriff.